

Data Structure(CSL202)

By-
Dr. Sk Subidh Ali (subidh@iitbhilai.ac.in)

Course Components

Introduction	Introduction to Data structures, Algorithm : Definition, characteristics, Classes, Efficiency
Asymptotic Analysis and Notations	Big-O Notation, Omega Notation (Ω-notation), Theta Notation (Θ-notation), Different properties of Asymptotic Notations,
Array	Linear Search, Binary Search, Bubble Sort, Insertion Sort, Merge Sort, Quick Sort
Linked List	Representation, Traversing, Searching, Insertion, Deletion, Circular Linked list, Doubly Linked List
Stack	Operations, Representation, Infix to Postfix, Postfix Expression Evaluation, Factorial Function, Fibonacci Sequence, Towers of Hanoi
Queue	Array Representation, Linked Representation, Deques, Priority Queues
Trees	Terminologies, Traversal, Binary Search Tree: Searching , Insertion, Deletion, AVL Tree, Heap, Heapsort
Graphs	Terminologies, Adjacency Matrix, Path Matrix, DFS, BFS, Topological Sorting

Course Tools & Platforms

Learning Management System (LMS):

- We use Canvas LMS for sharing lecture materials, announcements, quizzes, assignments, and grades.

Programming Language:

- All data structure implementations will be done using C Programming.

Code Editors & Compilers:

- **You may use any of the following based on your preference:**
  - **Ubuntu + GCC (GNU Compiler Collection):** Recommended for a Unix/Linux-based environment.
  - **Visual Studio Code (VSCode):** Cross-platform editor with C/C++ extensions.
  - **Turbo C:** Classic DOS-based IDE (use only if you're comfortable with it).

Grading Breakdown

Mid Semester	: 20%	
End Semester	: 40%	
Lab Test	: 20%	
Quizzes	:	
Home Assignments	:	20%
Lab Assignment	:	

Course Policies

- Don’t share your code with others.
- Don’t copy or look at someone else’s code, even if it’s just to “compare.”
- Do discuss concepts, strategies, and general approaches with peers.
- Do search online to understand ideas (but not to find exact solutions).
- Use of AI Tools (e.g., ChatGPT)
- Using tools like ChatGPT is allowed for inspiration or concept clarification.
- Submitting code you did not write or do not understand is academic misconduct.
- Always ensure your submission reflects your own understanding.

When in doubt, reach out! Talk to the instructor or TA for guidance.

Teaching Assistants

Deepesh Kumar Dwivedi(PhD Scholar in CSE, deepeshkd@iitbhilai.ac.in)

Shivani Sing (PhD scholar in DSAI)

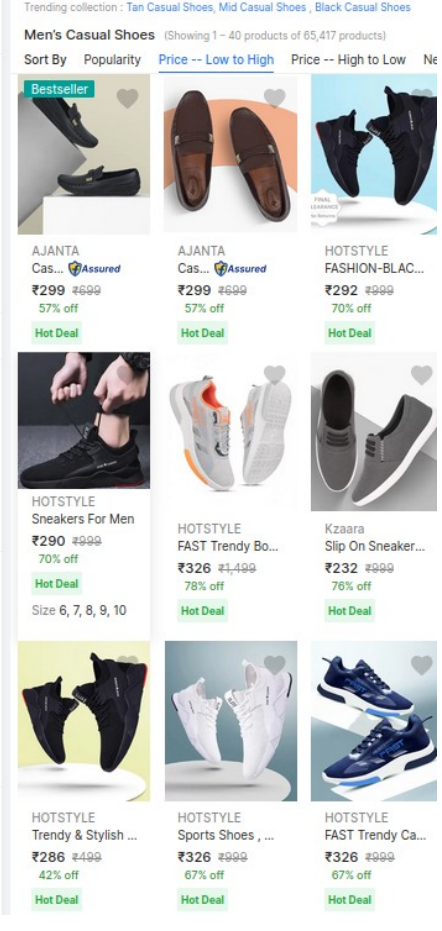
Reference Book:

- Source: Fundamentals of Data Structures in C by Ellis Horowitz
- Data Structures Schaum’s Outline Series
- Introduction to Algorithms by Thomas H. Cormen

Let me ask you few questions

- Have you ever browsing shoes on Flipkart?
- Its an application of :
  - Efficient Data structure
  - Sorting Algorithms
- Any idea how to do this?





Source of image: https://www.flipkart.com

Let me ask you few questions

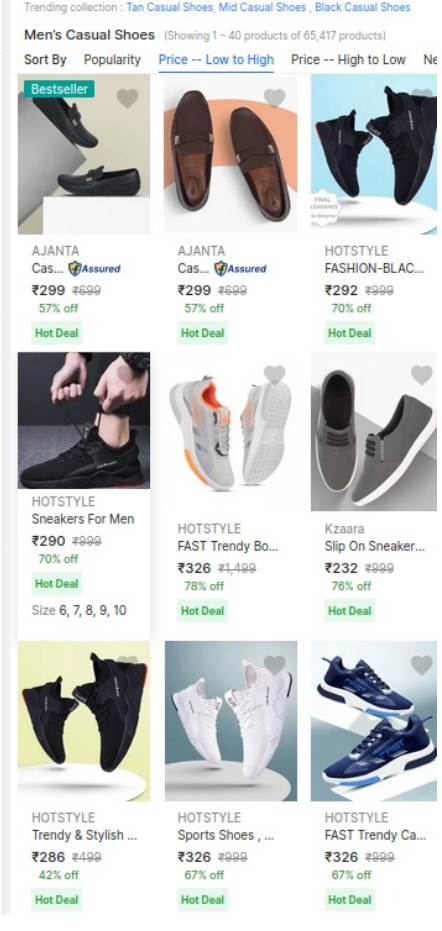
- Imagine you implement it in C

What database you have?



Brand	Type	Size	Price	Rating
Adidas	Running	8	5000	3.5
Nike	Running	7	8000	3
Reebok	Running	7	6000	3.7

Source of image: https://www.flipkart.com



Let me ask you few questions

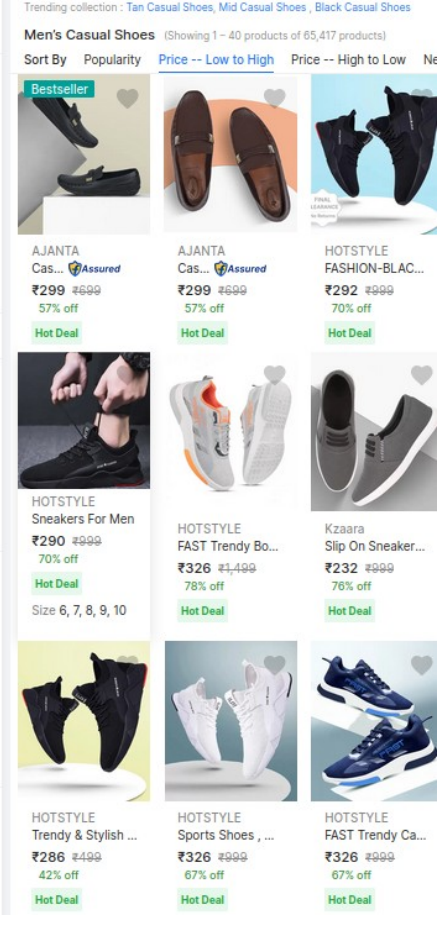
- Imagine you implement it in C

How to present the data?

Short as per the request



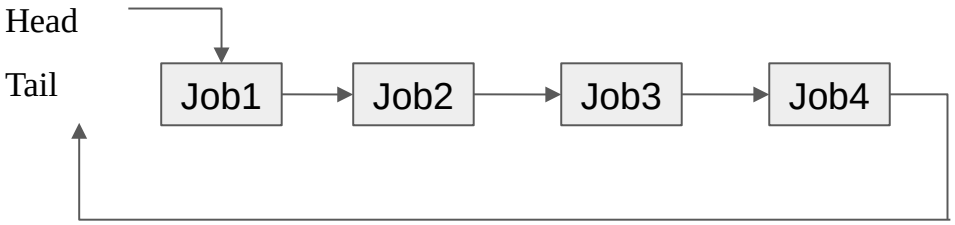
Brand	Type	Size	Price	Rating
Adidas	Running	8	5000	3.5
Nike	Running	7	8000	3
Reebok	Running	7	6000	3.7



Source of image: https://www.flipkart.com

Let me ask you few questions

- Have ever printed something on shared printer?



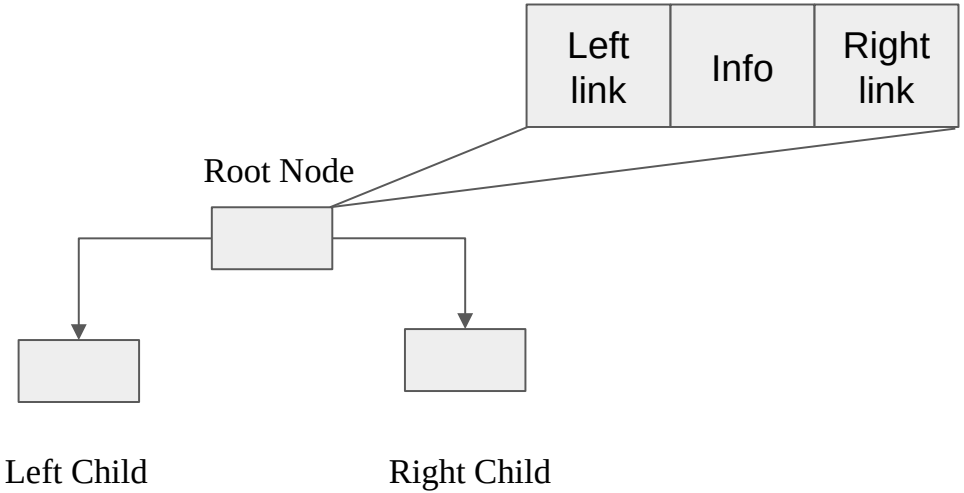


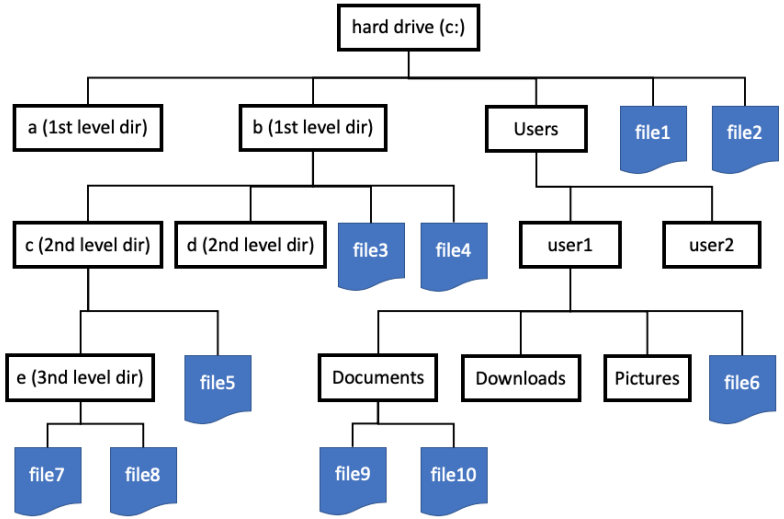
- Queue Data Structure.

Source : https://blog.thinprint.com/print-spooler-tasks-issues-high-availability/

Let me ask you few questions

- How Directory structure is organized in PC?





Source : https://heardlibrary.github.io/digital-scholarship/computer/directories-windows/

Let me ask you few questions

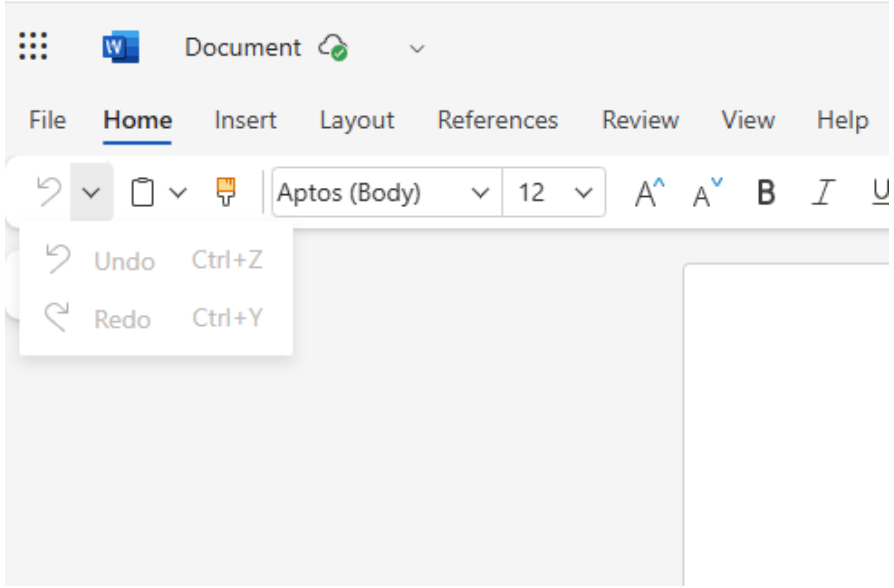
- How Ctrl+Z (undo it) is implemented in text-editors?

I am from Bhilai
I like coding
I am subidh

I am Subidh

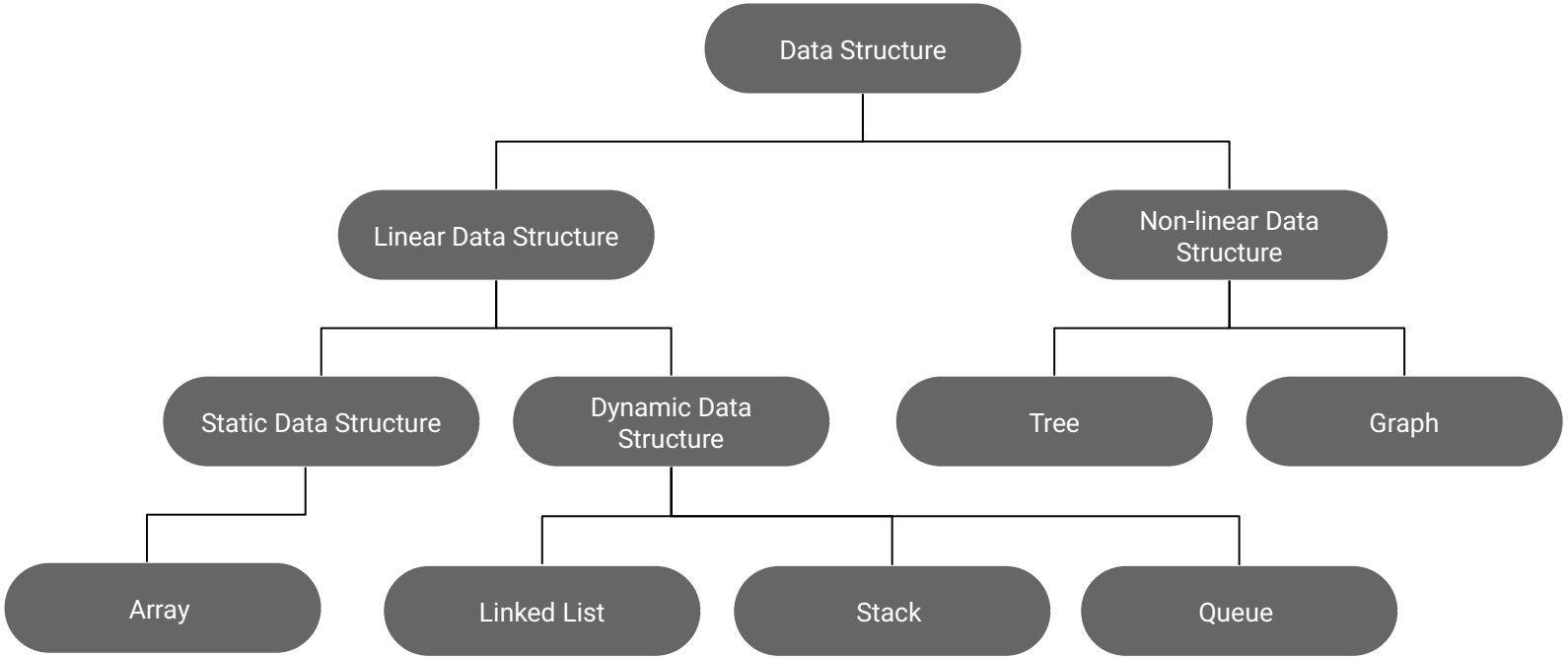
I like coding

I am from Bhilai



Data Structures

- The logical or mathematical model of a particular organization of data is called a data structure.



Data Structure Operations

- **Traverse:** Visit every element once (e.g., printing).
- **Search:** Find an element by key(e.g., Linear Search, Binary Search).
- **Insert:** Add a new element.
- **Delete:** Remove an existing element.
- **Sort:** Arrange elements in order (e.g., Selection Sort, Insertion Sort, Merge Sort, etc).
- **Merge:** Combine two sorted lists into one.



Algorithms

Algorithms

Algorithm

- An algorithm is a clearly specified sequence of simple instructions that solve a given problem.

Characteristics of an Algorithm

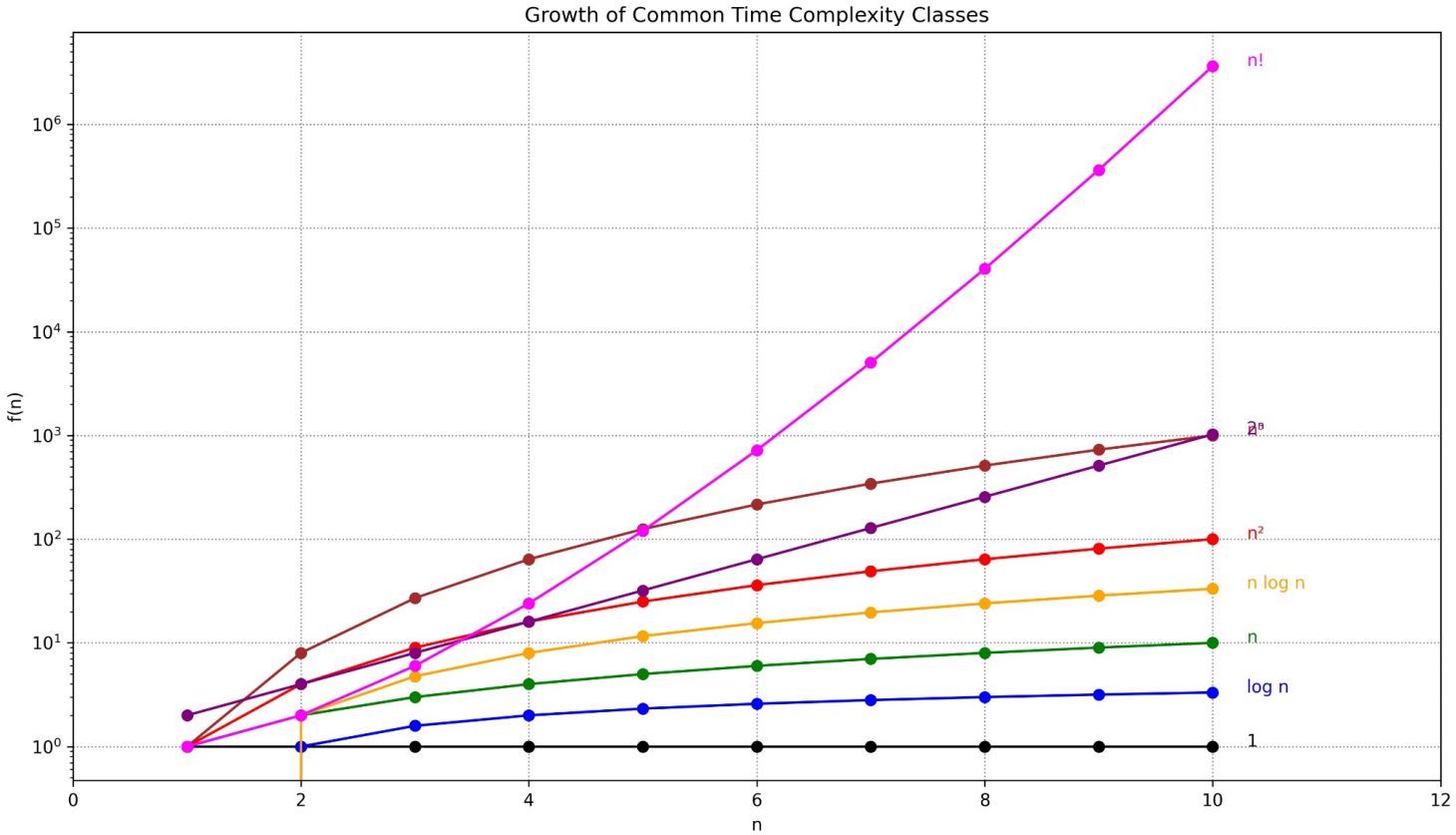
- **Input:** Should take zero or more input.
- **Output:** Should produce one or more outputs at the end of an algorithm.
- **Unambiguity:** The instructions in an algorithm should be clear and simple.
- **Finiteness:** Should contain a limited number of instructions or steps.
- **Effectiveness:** A human should be able to calculate the values involved in the procedure of the algorithm using pen and paper.
- **Termination:** An algorithm must terminate after a finite number of steps.

Classes of Algorithms

Class	Time	Growth of classes with n				
		1	2	4	8	16
Constant	1	1	1	1	1	1
Logarithmic	log n	0	1	2	3	4
Linear	n	1	2	4	8	16
Log linear	n log n	0	2	8	24	64
Quadratic	n^2	1	4	16	64	256
Cubic	n^3	1	8	64	512	4096
Exponential	2^n	2	4	16	256	65536
Factorial	n!	1	2	24	40326	20922789888000

Source: Fundamentals of Data Structures in C by Ellis Horowitz

Classes of Algorithms



Algorithm Efficiency

- For the same problem, there could be multiple algorithms.
- By analysing different algorithms, we can compare them to determine the best one for our purpose.
- In order to compare algorithms we must have some criteria to measure the efficiency of our algorithms.
- The complexity of an algorithm is the function f(n) gives the running time or storage space required by the algorithm in terms of size n of input data.

Two main measures for the efficiency of algorithm are:

- **Time complexity-** Amount of time it needs to run for its completion.
- **Space complexity-** Amount of memory it needs to run for its completion.

Examples

```
a = a + b;
b = a - b;
a = a - b;
```

```
a[5] = 6;
```

```
for (i=0;i<100;i++)
        A[i] = 6;
```

```
x = y;
if (x > 0)
        y = x + 1;
else
        z = x + 1;
```

- All of these are equally efficient!
- They all perform constant-time operations.
- We denote those as O(1).

Examples

```
int fact(int n)
        return
n*fact(n-1);
```

```
for (i=0;i<n;i++)
        A[i] = 6;
```

```
a [0] = 0;
a [1] = 2;
a [2] = 6;
. . . .
. . . .
a [n-1] = 0;
```

- All of these are equally efficient!
- They all perform linear-time operations.
- We denote those as O(n).